

Mẫu hình học tính toán

Ji Ruyi

Nguồn gốc: Computational Geometry Template

```
#define mp make_pair
#define fi first
#define se second
#define pb push_back
typedef double db;
const db eps = 1e-6;
const db pi = acos(-1);
int sign(db k) {
    if (k > eps) return 1; else if (k < -eps) return -1; return 0;
}
int cmp(db k1, db k2) {return sign(k1 - k2);}
int inmid(db k1, db k2, db k3) {return sign(k1 - k3) * sign(k2 - k3) <= 0;} // k3 nằm trong [k1,k2]
struct point {
    db x, y;
    point operator + (const point &k1) const {return (point) {k1.x + x, k1.y + y};}
    point operator - (const point &k1) const {return (point) {x - k1.x, y - k1.y};}
    point operator * (db k1) const {return (point) {x*k1, y*k1};}
    point operator / (db k1) const {return (point) {x / k1, y / k1};}
    int operator == (const point &k1) const {return cmp(x, k1.x) == 0 && cmp(y, k1.y) == 0;}
    // Xoay ngược chiều kim đồng hồ
    point turn(db k1) {return (point) {x*cos(k1) - y*sin(k1), x*sin(k1) + y*cos(k1)};}
    point turn90() {return (point) {-y, x};}
    bool operator < (const point k1) const {
        int a = cmp(x, k1.x);
        if (a == -1) return 1; else if (a == 1) return 0; else return cmp(y, k1.y) == -1;
    }
    db abs() {return sqrt(x * x + y * y);}
    db abs2() {return x * x + y * y;}
    db dis(point k1) {return ((*this) - k1).abs();}
    point unit() {db w = abs(); return (point) {x / w, y / w};}
    void scan() {double k1, k2; scanf("%lf%lf", &k1, &k2); x = k1; y = k2;}
    void print() {printf("%.11lf %.11lf\n", x, y);}
    db getw() {return atan2(y, x);}
    point getdel() {if (sign(x) == -1 || (sign(x) == 0 && sign(y) == -1)) return (*this) * (-1); else return (*this);}
    int getP() const {return sign(y) == 1 || (sign(y) == 0 && sign(x) == -1);}
};
int inmid(point k1, point k2, point k3) {return inmid(k1.x, k2.x, k3.x) && inmid(k1.y, k2.y, k3.y);}
db cross(point k1, point k2) {return k1.x * k2.y - k1.y * k2.x;}
db dot(point k1, point k2) {return k1.x * k2.x + k1.y * k2.y;}
db rad(point k1, point k2) {return atan2(cross(k1, k2), dot(k1, k2));}
// -pi -> pi
int compareangle (point k1, point k2) {
    return k1.getP() < k2.getP() || (k1.getP() == k2.getP() && sign(cross(k1, k2)) > 0);
}
point proj(point k1, point k2, point q) { // Hình chiếu của q lên đường thẳng k1,k2
    point k = k2 - k1; return k1 + k * (dot(q - k1, k) / k.abs2());
}
point reflect(point k1, point k2, point q) {return proj(k1, k2, q) * 2 - q;}
int clockwise(point k1, point k2, point k3) { // k1 k2 k3: ngược chiều kim đồng hồ 1, cùng chiều -1, ngược lại 0
    return sign(cross(k2 - k1, k3 - k1));
}
int checkLL(point k1, point k2, point k3, point k4) { // Tìm giao điểm của đường thẳng (L) / đoạn thẳng (S) k1,k2 và k3,k4
    return cmp(cross(k3 - k1, k4 - k1), cross(k3 - k2, k4 - k2)) != 0;
}
```

```

point getLL(point k1, point k2, point k3, point k4) {
    db w1 = cross(k1 - k3, k4 - k3), w2 = cross(k4 - k3, k2 - k3); return (k1 * w2 + k2 * w1) / (w1 + w2);
}
int intersect(db l1, db r1, db l2, db r2) {
    if (l1 > r1) swap(l1, r1); if (l2 > r2) swap(l2, r2); return cmp(r1, l2) != -1 && cmp(r2, l1) != -1;
}
int checkSS(point k1, point k2, point k3, point k4) {
    return intersect(k1.x, k2.x, k3.x, k4.x) && intersect(k1.y, k2.y, k3.y, k4.y) &&
        sign(cross(k3 - k1, k4 - k1)) * sign(cross(k3 - k2, k4 - k2)) <= 0 &&
        sign(cross(k1 - k3, k2 - k3)) * sign(cross(k1 - k4, k2 - k4)) <= 0;
}
db disSP(point k1, point k2, point q) {
    point k3 = proj(k1, k2, q);
    if (inmid(k1, k2, k3)) return q.dis(k3); else return min(q.dis(k1), q.dis(k2));
}
db disSS(point k1, point k2, point k3, point k4) {
    if (checkSS(k1, k2, k3, k4)) return 0;
    else return min(min(disSP(k1, k2, k3), disSP(k1, k2, k4)), min(disSP(k3, k4, k1), disSP(k3, k4, k2)));
}
int onS(point k1, point k2, point q) {return inmid(k1, k2, q) && sign(cross(k1 - q, k2 - k1)) == 0;}
struct circle {
    point o; db r;
    void scan() {o.scan(); scanf("%lf", &r);}
    int inside(point k) {return cmp(r, o.dis(k));}
};
struct line {
    // p[0]->p[1]
    point p[2];
    line(point k1, point k2) {p[0] = k1; p[1] = k2;}
    point& operator [] (int k) {return p[k];}
    int include(point k) {return sign(cross(p[1] - p[0], k - p[0])) > 0;}
    point dir() {return p[1] - p[0];}
    line push() { // Dịch ra ngoài (phía tay trái) một đoạn eps
        const db eps = 1e-6;
        point delta = (p[1] - p[0]).turn90().unit() * eps;
        return {p[0] - delta, p[1] - delta};
    }
};
point getLL(line k1, line k2) {return getLL(k1[0], k1[1], k2[0], k2[1]);}
int parallel(line k1, line k2) {return sign(cross(k1.dir(), k2.dir())) == 0;}
int sameDir(line k1, line k2) {return parallel(k1, k2) && sign(dot(k1.dir(), k2.dir())) == 1;}
int operator < (line k1, line k2) {
    if (sameDir(k1, k2)) return k2.include(k1[0]);
    return compareangle(k1.dir(), k2.dir());
}
int checkpos(line k1, line k2, line k3) {return k3.include(getLL(k1, k2));}
vector<line> getHL(vector<line> &L) { // Tìm giao nửa mặt phẳng; nửa mặt phẳng có hướng ngược chiều kim đồng hồ, xuất theo thứ tự
    sort(L.begin(), L.end()); deque<line> q;
    for (int i = 0; i < (int)L.size(); i++) {
        if (i && sameDir(L[i], L[i - 1])) continue;
        while (q.size() > 1 && !checkpos(q[q.size() - 2], q[q.size() - 1], L[i])) q.pop_back();
        while (q.size() > 1 && !checkpos(q[1], q[0], L[i])) q.pop_front();
        q.push_back(L[i]);
    }
    while (q.size() > 2 && !checkpos(q[q.size() - 2], q[q.size() - 1], q[0])) q.pop_back();
    while (q.size() > 2 && !checkpos(q[1], q[0], q[q.size() - 1])) q.pop_front();
    vector<line> ans; for (int i = 0; i < q.size(); i++) ans.push_back(q[i]);
    return ans;
}
db closepoint(vector<point>&A, int l, int r) { // Cặp điểm gần nhất; cần sắp xếp trước theo tọa độ x
    if (r - l <= 5) {
        db ans = 1e20;
        for (int i = l; i <= r; i++) for (int j = i + 1; j <= r; j++) ans = min(ans, A[i].dis(A[j]));
        return ans;
    }
    int mid = l + r >> 1; db ans = min(closepoint(A, l, mid), closepoint(A, mid + 1, r));
    vector<point>B; for (int i = l; i <= r; i++) if (abs(A[i].x - A[mid].x) <= ans) B.push_back(A[i]);
    sort(B.begin(), B.end(), [](point k1, point k2) {return k1.y < k2.y;});
    for (int i = 0; i < B.size(); i++) for (int j = i + 1; j < B.size() && B[j].y - B[i].y < ans; j++) ans = min(ans, B[i].dis(B[j]));
}

```

```

    return ans;
}
int checkposCC(circle k1, circle k2) { // Trả về số tiếp tuyến chung của hai đường tròn
    if (cmp(k1.r, k2.r) == -1) swap(k1, k2);
    db dis = k1.o.dis(k2.o); int w1 = cmp(dis, k1.r + k2.r), w2 = cmp(dis, k1.r - k2.r);
    if (w1 > 0) return 4; else if (w1 == 0) return 3; else if (w2 > 0) return 2;
    else if (w2 == 0) return 1; else return 0;
}
vector<point> getCL(circle k1, point k2, point k3) { // Trả về theo hướng k2->k3; nếu tiếp xúc thì trả về hai điểm
    point k = proj(k2, k3, k1.o); db d = k1.r * k1.r - (k - k1.o).abs2();
    if (sign(d) == -1) return {};
    point del = (k3 - k2).unit() * sqrt(max((db)0.0, d)); return {k - del, k + del};
}
vector<point> getCC(circle k1, circle k2) { // Trả về theo chiều ngược kim đồng hồ trên đường tròn k1; nếu tiếp xúc thì trả về hai
    int pd = checkposCC(k1, k2); if (pd == 0 || pd == 4) return {};
    db a = (k2.o - k1.o).abs2(), cosA = (k1.r * k1.r + a - k2.r * k2.r) / (2 * k1.r * sqrt(max(a, (db)0.0)));
    db b = k1.r * cosA, c = sqrt(max((db)0.0, k1.r * k1.r - b * b));
    point k = (k2.o - k1.o).unit(), m = k1.o + k * b, del = k.turn90() * c;
    return {m - del, m + del};
}
vector<point> TangentCP(circle k1, point k2) { // Trả về theo chiều ngược kim đồng hồ trên đường tròn k1
    db a = (k2 - k1.o).abs(), b = k1.r * k1.r / a, c = sqrt(max((db)0.0, k1.r * k1.r - b * b));
    point k = (k2 - k1.o).unit(), m = k1.o + k * b, del = k.turn90() * c;
    return {m - del, m + del};
}
vector<line> TangentoutCC(circle k1, circle k2) {
    int pd = checkposCC(k1, k2); if (pd == 0) return {};
    if (pd == 1) {point k = getCC(k1, k2)[0]; return {(line) {k, k}};}
    if (cmp(k1.r, k2.r) == 0) {
        point del = (k2.o - k1.o).unit().turn90().getdel();
        return {(line){k1.o - del * k1.r, k2.o - del * k2.r}, (line) {k1.o + del*k1.r, k2.o + del*k2.r}};
    } else {
        point p = (k2.o * k1.r - k1.o * k2.r) / (k1.r - k2.r);
        vector<point>A = TangentCP(k1, p), B = TangentCP(k2, p);
        vector<line>ans; for (int i = 0; i < A.size(); i++) ans.push_back((line) {A[i], B[i]});
        return ans;
    }
}
vector<line> TangentinCC(circle k1, circle k2) {
    int pd = checkposCC(k1, k2); if (pd <= 2) return {};
    if (pd == 3) {point k = getCC(k1, k2)[0]; return {(line) {k, k}};}
    point p = (k2.o * k1.r + k1.o * k2.r) / (k1.r + k2.r);
    vector<point>A = TangentCP(k1, p), B = TangentCP(k2, p);
    vector<line>ans; for (int i = 0; i < A.size(); i++) ans.push_back((line) {A[i], B[i]});
    return ans;
}
vector<line> TangentCC(circle k1, circle k2) {
    int flag = 0; if (k1.r < k2.r) swap(k1, k2), flag = 1;
    vector<line>A = TangentoutCC(k1, k2), B = TangentinCC(k1, k2);
    for (line k : B) A.push_back(k);
    if (flag) for (line &k : A) swap(k[0], k[1]);
    return A;
}
db getarea(circle k1, point k2, point k3) {
    // Diện tích có hướng của phần giao giữa đường tròn k1 và tam giác k2 k3 k1.o
    point k = k1.o; k1.o = k1.o - k; k2 = k2 - k; k3 = k3 - k;
    int pd1 = k1.inside(k2), pd2 = k1.inside(k3);
    vector<point>A = getCL(k1, k2, k3);
    if (pd1 >= 0) {
        if (pd2 >= 0) return cross(k2, k3) / 2;
        return k1.r * k1.r * rad(A[1], k3) / 2 + cross(k2, A[1]) / 2;
    } else if (pd2 >= 0) {
        return k1.r * k1.r * rad(k2, A[0]) / 2 + cross(A[0], k3) / 2;
    } else {
        int pd = cmp(k1.r, disSP(k2, k3, k1.o));
        if (pd <= 0) return k1.r * k1.r * rad(k2, k3) / 2;
        return cross(A[0], A[1]) / 2 + k1.r * k1.r * (rad(k2, A[0]) + rad(A[1], k3)) / 2;
    }
}
}

```

```

circle getcircle(point k1, point k2, point k3) {
    db a1 = k2.x - k1.x, b1 = k2.y - k1.y, c1 = (a1 * a1 + b1 * b1) / 2;
    db a2 = k3.x - k1.x, b2 = k3.y - k1.y, c2 = (a2 * a2 + b2 * b2) / 2;
    db d = a1 * b2 - a2 * b1;
    point o = (point) {k1.x + (c1 * b2 - c2 * b1) / d, k1.y + (a1 * c2 - a2 * c1) / d};
    return (circle) {o, k1.dis(o)};
}

circle getScircle(vector<point> A) {
    random_shuffle(A.begin(), A.end());
    circle ans = (circle) {A[0], 0};
    for (int i = 1; i < A.size(); i++)
        if (ans.inside(A[i]) == -1) {
            ans = (circle) {A[i], 0};
            for (int j = 0; j < i; j++)
                if (ans.inside(A[j]) == -1) {
                    ans.o = (A[i] + A[j]) / 2; ans.r = ans.o.dis(A[i]);
                    for (int k = 0; k < j; k++)
                        if (ans.inside(A[k]) == -1)
                            ans = getcircle(A[i], A[j], A[k]);
                }
        }
    return ans;
}

db area(vector<point> A) { // Đa giác biểu diễn bằng vector<point>, theo chiều ngược kim đồng hồ
    db ans = 0;
    for (int i = 0; i < A.size(); i++) ans += cross(A[i], A[(i + 1) % A.size()]);
    return ans / 2;
}

int checkconvex(vector<point>A) {
    int n = A.size(); A.push_back(A[0]); A.push_back(A[1]);
    for (int i = 0; i < n; i++) if (sign(cross(A[i + 1] - A[i], A[i + 2] - A[i])) == -1) return 0;
    return 1;
}

int contain(vector<point>A, point q) { // 2 bên trong, 1 biên, 0 bên ngoài
    int pd = 0; A.push_back(A[0]);
    for (int i = 1; i < A.size(); i++) {
        point u = A[i - 1], v = A[i];
        if (onS(u, v, q)) return 1; if (cmp(u.y, v.y) > 0) swap(u, v);
        if (cmp(u.y, q.y) >= 0 || cmp(v.y, q.y) < 0) continue;
        if (sign(cross(u - v, q - v)) < 0) pd ^= 1;
    }
    return pd << 1;
}

vector<point> ConvexHull(vector<point>A, int flag = 1) { // flag=0 không chặt, flag=1 chặt
    int n = A.size(); vector<point>ans(n * 2);
    sort(A.begin(), A.end()); int now = -1;
    for (int i = 0; i < A.size(); i++) {
        while (now > 0 && sign(cross(ans[now] - ans[now - 1], A[i] - ans[now - 1])) < flag) now--;
        ans[++now] = A[i];
    } int pre = now;
    for (int i = n - 2; i >= 0; i--) {
        while (now > pre && sign(cross(ans[now] - ans[now - 1], A[i] - ans[now - 1])) < flag) now--;
        ans[++now] = A[i];
    } ans.resize(now); return ans;
}

db convexDiameter(vector<point>A) {
    int now = 0, n = A.size(); db ans = 0;
    for (int i = 0; i < A.size(); i++) {
        now = max(now, i);
        while (1) {
            db k1 = A[i].dis(A[now % n]), k2 = A[i].dis(A[(now + 1) % n]);
            ans = max(ans, max(k1, k2)); if (k2 > k1) now++; else break;
        }
    }
    return ans;
}

vector<point> convexcut(vector<point>A, point k1, point k2) {
    // Giữ lại tất cả các điểm sao cho k1,k2,p theo chiều ngược kim đồng hồ
    int n = A.size(); A.push_back(A[0]); vector<point>ans;

```

```

for (int i = 0; i < n; i++) {
    int w1 = clockwise(k1, k2, A[i]), w2 = clockwise(k1, k2, A[i + 1]);
    if (w1 >= 0) ans.push_back(A[i]);
    if (w1 * w2 < 0) ans.push_back(getLL(k1, k2, A[i], A[i + 1]));
}
return ans;
}

int checkPoS(vector<point>A, point k1, point k2) {
    // Đa giác A và đường thẳng (đoạn thẳng) k1->k2 giao nhau chặt; phần chú thích là cho đoạn thẳng
    struct ins {
        point m, u, v;
        int operator < (const ins& k) const {return m < k.m;}
    }; vector<ins>B;
    //if (contain(A,k1)==2||contain(A,k2)==2) return 1;
    vector<point>poly = A; A.push_back(A[0]);
    for (int i = 1; i < A.size(); i++) if (checkLL(A[i - 1], A[i], k1, k2)) {
        point m = getLL(A[i - 1], A[i], k1, k2);
        if (inmid(A[i - 1], A[i], m)/*&&inmid(k1,k2,m)*/) B.push_back((ins) {m, A[i - 1], A[i]});
    }
    if (B.size() == 0) return 0; sort(B.begin(), B.end());
    int now = 1; while (now < B.size() && B[now].m == B[0].m) now++;
    if (now == B.size()) return 0;
    int flag = contain(poly, (B[0].m + B[now].m) / 2);
    if (flag == 2) return 1;
    point d = B[now].m - B[0].m;
    for (int i = now; i < B.size(); i++) {
        if (!(B[i].m == B[i - 1].m) && flag == 2) return 1;
        int tag = sign(cross(B[i].v - B[i].u, B[i].m + d - B[i].u));
        if (B[i].m == B[i].u || B[i].m == B[i].v) flag += tag; else flag += tag * 2;
    }
    //return 0;
    return flag == 2;
}

int checkinp(point r, point l, point m) {
    if (compareangle(l, r)) {return compareangle(l, m) && compareangle(m, r);}
    return compareangle(l, m) || compareangle(m, r);
}

int checkPosFast(vector<point>A, point k1, point k2) { // Kiểm tra nhanh đoạn thẳng có giao chặt với đa giác hay không
    if (contain(A, k1) == 2 || contain(A, k2) == 2) return 1; if (k1 == k2) return 0;
    A.push_back(A[0]); A.push_back(A[1]);
    for (int i = 1; i + 1 < A.size(); i++)
        if (checkLL(A[i - 1], A[i], k1, k2)) {
            point now = getLL(A[i - 1], A[i], k1, k2);
            if (inmid(A[i - 1], A[i], now) == 0 || inmid(k1, k2, now) == 0) continue;
            if (now == A[i]) {
                if (A[i] == k2) continue;
                point pre = A[i - 1], ne = A[i + 1];
                if (checkinp(pre - now, ne - now, k2 - now)) return 1;
            } else if (now == k1) {
                if (k1 == A[i - 1] || k1 == A[i]) continue;
                if (checkinp(A[i - 1] - k1, A[i] - k1, k2 - k1)) return 1;
            } else if (now == k2 || now == A[i - 1]) continue;
            else return 1;
        }
    return 0;
}

// Tách bao lồi thành vỏ lồi trên và vỏ lồi dưới. Nên xoay ngẫu nhiên bao lồi một góc để tránh các hoành độ trùng nhau.
// Nên xử lý riêng trường hợp chỉ có một điểm. Bao lồi theo chiều ngược kim đồng hồ.
void getUDP(vector<point>A, vector<point>&U, vector<point>&D) {
    db l = 1e100, r = -1e100;
    for (int i = 0; i < A.size(); i++) l = min(l, A[i].x), r = max(r, A[i].x);
    int wherel, wherer;
    for (int i = 0; i < A.size(); i++) if (cmp(A[i].x, l) == 0) wherel = i;
    for (int i = A.size(); i; i--) if (cmp(A[i - 1].x, r) == 0) wherer = i - 1;
    U.clear(); D.clear(); int now = wherel;
    while (1) {D.push_back(A[now]); if (now == wherer) break; now++; if (now >= A.size()) now = 0;}
    now = wherel;
    while (1) {U.push_back(A[now]); if (now == wherer) break; now--; if (now < 0) now = A.size() - 1;}
}

```

```

// Cần đảm bảo bao lồi có số điểm >= 3; 2 bên trong, 1 biên, 0 bên ngoài
int containCoP(const vector<point>&U, const vector<point>&D, point k) {
    db lx = U[0].x, rx = U[U.size() - 1].x;
    if (k == U[0] || k == U[U.size() - 1]) return 1;
    if (cmp(k.x, lx) == -1 || cmp(k.x, rx) == 1) return 0;
    int where1 = lower_bound(U.begin(), U.end(), (point) {k.x, -1e100}) - U.begin();
    int where2 = lower_bound(D.begin(), D.end(), (point) {k.x, -1e100}) - D.begin();
    int w1 = clockwise(U[where1 - 1], U[where1], k), w2 = clockwise(D[where2 - 1], D[where2], k);
    if (w1 == 1 || w2 == -1) return 0; else if (w1 == 0 || w2 == 0) return 1; return 2;
}

// d là hướng; trả về tiếp điểm phía trên và tiếp điểm phía dưới
pair<point, point> getTangentCow(const vector<point>&U, const vector<point>&D, point d) {
    if (sign(d.x) < 0 || (sign(d.x) == 0 && sign(d.y) < 0)) d = d * (-1);
    point whereU, whereD;
    if (sign(d.x) == 0) return mp(U[0], U[U.size() - 1]);
    int l = 0, r = U.size() - 1, ans = 0;
    while (l < r) {int mid = l + r >> 1; if (sign(cross(U[mid + 1] - U[mid], d)) <= 0) l = mid + 1, ans = mid + 1; else r = mid;}
    whereU = U[ans]; l = 0, r = D.size() - 1, ans = 0;
    while (l < r) {int mid = l + r >> 1; if (sign(cross(D[mid + 1] - D[mid], d)) >= 0) l = mid + 1, ans = mid + 1; else r = mid;}
    whereD = D[ans]; return mp(whereU, whereD);
}

// Kiểm tra contain trước; trả về theo chiều ngược kim đồng hồ
pair<point, point> getTangentCoP(const vector<point>&U, const vector<point>&D, point k) {
    db lx = U[0].x, rx = U[U.size() - 1].x;
    if (k.x < lx) {
        int l = 0, r = U.size() - 1, ans = U.size() - 1;
        while (l < r) {int mid = l + r >> 1; if (clockwise(k, U[mid], U[mid + 1]) == 1) l = mid + 1; else ans = mid, r = mid;}
        point w1 = U[ans]; l = 0, r = D.size() - 1, ans = D.size() - 1;
        while (l < r) {int mid = l + r >> 1; if (clockwise(k, D[mid], D[mid + 1]) == -1) l = mid + 1; else ans = mid, r = mid;}
        point w2 = D[ans]; return mp(w1, w2);
    } else if (k.x > rx) {
        int l = 1, r = U.size(), ans = 0;
        while (l < r) {int mid = l + r >> 1; if (clockwise(k, U[mid], U[mid - 1]) == -1) r = mid; else ans = mid, l = mid + 1;}
        point w1 = U[ans]; l = 1, r = D.size(), ans = 0;
        while (l < r) {int mid = l + r >> 1; if (clockwise(k, D[mid], D[mid - 1]) == 1) r = mid; else ans = mid, l = mid + 1;}
        point w2 = D[ans]; return mp(w2, w1);
    } else {
        int where1 = lower_bound(U.begin(), U.end(), (point) {k.x, -1e100}) - U.begin();
        int where2 = lower_bound(D.begin(), D.end(), (point) {k.x, -1e100}) - D.begin();
        if ((k.x == lx && k.y > U[0].y) || (where1 && clockwise(U[where1 - 1], U[where1], k) == 1)) {
            int l = 1, r = where1 + 1, ans = 0;
            while (l < r) {int mid = l + r >> 1; if (clockwise(k, U[mid], U[mid - 1]) == 1) ans = mid, l = mid + 1; else r = mid;}
            point w1 = U[ans]; l = where1, r = U.size() - 1, ans = U.size() - 1;
            while (l < r) {int mid = l + r >> 1; if (clockwise(k, U[mid], U[mid + 1]) == 1) l = mid + 1; else ans = mid, r = mid;}
            point w2 = U[ans]; return mp(w2, w1);
        } else {
            int l = 1, r = where2 + 1, ans = 0;
            while (l < r) {int mid = l + r >> 1; if (clockwise(k, D[mid], D[mid - 1]) == -1) ans = mid, l = mid + 1; else r = mid;}
            point w1 = D[ans]; l = where2, r = D.size() - 1, ans = D.size() - 1;
            while (l < r) {int mid = l + r >> 1; if (clockwise(k, D[mid], D[mid + 1]) == -1) l = mid + 1; else ans = mid, r = mid;}
            point w2 = D[ans]; return mp(w1, w2);
        }
    }
}

struct P3 {
    db x, y, z;
    P3 operator + (P3 k1) {return (P3) {x + k1.x, y + k1.y, z + k1.z};}
    P3 operator - (P3 k1) {return (P3) {x - k1.x, y - k1.y, z - k1.z};}
    P3 operator * (db k1) {return (P3) {x*k1, y*k1, z*k1};}
    P3 operator / (db k1) {return (P3) {x / k1, y / k1, z / k1};}
    db abs2() {return x * x + y * y + z * z;}
    db abs() {return sqrt(x * x + y * y + z * z);}
    P3 unit() {return (*this) / abs();}
    int operator < (const P3 k1) const {
        if (cmp(x, k1.x) != 0) return x < k1.x;
        if (cmp(y, k1.y) != 0) return y < k1.y;
        return cmp(z, k1.z) == -1;
    }
}

int operator == (const P3 k1) {

```

```

    return cmp(x, k1.x) == 0 && cmp(y, k1.y) == 0 && cmp(z, k1.z) == 0;
}
void scan() {
    double k1, k2, k3; scanf("%lf%lf%lf", &k1, &k2, &k3);
    x = k1; y = k2; z = k3;
}
};
P3 cross(P3 k1, P3 k2) {return (P3) {k1.y*k2.z - k1.z*k2.y, k1.z*k2.x - k1.x*k2.z, k1.x*k2.y - k1.y*k2.x};}
db dot(P3 k1, P3 k2) {return k1.x * k2.x + k1.y * k2.y + k1.z * k2.z;}
//p=(3,4,5),l=(13,19,21),theta=85 ans=(2.83,4.62,1.77)
P3 turn3D(db k1, P3 l, P3 p) {
    l = l.unit(); P3 ans; db c = cos(k1), s = sin(k1);
    ans.x = p.x * (l.x * l.x * (1 - c) + c) + p.y * (l.x * l.y * (1 - c) - l.z * s) + p.z * (l.x * l.z * (1 - c) + l.y * s);
    ans.y = p.x * (l.x * l.y * (1 - c) + l.z * s) + p.y * (l.y * l.y * (1 - c) + c) + p.z * (l.y * l.z * (1 - c) - l.x * s);
    ans.z = p.x * (l.x * l.z * (1 - c) - l.y * s) + p.y * (l.y * l.z * (1 - c) + l.x * s) + p.z * (l.x * l.x * (1 - c) + c);
    return ans;
}
typedef vector<P3> VP;
typedef vector<VP> VVP;
db Acos(db x) {return acos(max(-(db)1, min(x, (db)1)));}
// Khoảng cách trên mặt cầu, tâm là gốc tọa độ, bán kính 1
db Odist(P3 a, P3 b) {db r = Acos(dot(a, b)); return r;}
db r; P3 rnd;
vector<db> solve(db a, db b, db c) {
    db r = sqrt(a * a + b * b), th = atan2(b, a);
    if (cmp(c, -r) == -1) return {0};
    else if (cmp(r, c) <= 0) return {1};
    else {
        db tr = pi - Acos(c / r); return {th + pi - tr, th + pi + tr};
    }
}
vector<db> jiao(P3 a, P3 b) {
    // dot(rd+x*cos(t)+y*sin(t),b) >= cos(r)
    if (cmp(Odist(a, b), 2 * r) > 0) return {0};
    P3 rd = a * cos(r), z = a.unit(), y = cross(z, rnd).unit(), x = cross(y, z).unit();
    vector<db> ret = solve(-(dot(x, b) * sin(r)), -(dot(y, b) * sin(r)), -(cos(r) - dot(rd, b)));
    return ret;
}
db norm(db x, db l = 0, db r = 2 * pi) { // change x into [l,r)
    while (cmp(x, l) == -1) x += (r - l); while (cmp(x, r) >= 0) x -= (r - l);
    return x;
}
db disLP(P3 k1, P3 k2, P3 q) {
    return (cross(k2 - k1, q - k1)).abs() / (k2 - k1).abs();
}
db disLL(P3 k1, P3 k2, P3 k3, P3 k4) {
    P3 dir = cross(k2 - k1, k4 - k3); if (sign(dir.abs()) == 0) return disLP(k1, k2, k3);
    return fabs(dot(dir.unit(), k1 - k2));
}
VP getFL(P3 p, P3 dir, P3 k1, P3 k2) {
    db a = dot(k2 - p, dir), b = dot(k1 - p, dir), d = a - b;
    if (sign(fabs(d)) == 0) return {};
    return {(k1 * a - k2 * b) / d};
}
VP getFF(P3 p1, P3 dir1, P3 p2, P3 dir2) { // Trả về một đường thẳng
    P3 e = cross(dir1, dir2), v = cross(dir1, e);
    db d = dot(dir2, v); if (sign(fabs(d)) == 0) return {};
    P3 q = p1 + v * dot(dir2, p2 - p1) / d; return {q, q + e};
}
// 3D Convex Hull Template
db getV(P3 k1, P3 k2, P3 k3, P3 k4) { // Tính thể tích
    return dot(cross(k2 - k1, k3 - k1), k4 - k1);
}
db rand_db() {return 1.0 * rand() / RAND_MAX;}
VP convexHull2D(VP A, P3 dir) {
    P3 x = {(db)rand(), (db)rand(), (db)rand()}; x = x.unit();
    x = cross(x, dir).unit(); P3 y = cross(x, dir).unit();
    P3 vec = dir.unit() * dot(A[0], dir);
    vector<point>B;

```

```

for (int i = 0; i < A.size(); i++) B.push_back((point) {dot(A[i], x), dot(A[i], y)});
B = ConvexHull(B); A.clear();
for (int i = 0; i < B.size(); i++) A.push_back(x * B[i].x + y * B[i].y + vec);
return A;
}
namespace CH3 {
VVP ret; set<pair<int, int> >e;
int n; VP p, q;
void wrap(int a, int b) {
if (e.find({a, b}) == e.end()) {
int c = -1;
for (int i = 0; i < n; i++) if (i != a && i != b) {
if (c == -1 || sign(getV(q[c], q[a], q[b], q[i])) > 0) c = i;
}
if (c != -1) {
ret.push_back({p[a], p[b], p[c]});
e.insert({a, b}); e.insert({b, c}); e.insert({c, a});
wrap(c, b); wrap(a, c);
}
}
}
VVP ConvexHull3D(VP _p) {
p = q = _p; n = p.size();
ret.clear(); e.clear();
for (auto &i : q) i = i + (P3) {rand_db() * 1e-4, rand_db() * 1e-4, rand_db() * 1e-4};
for (int i = 1; i < n; i++) if (q[i].x < q[0].x) swap(p[0], p[i]), swap(q[0], q[i]);
for (int i = 2; i < n; i++) if ((q[i].x - q[0].x) * (q[1].y - q[0].y) > (q[i].y - q[0].y) * (q[1].x - q[0].x)) swap(q[1], q[i]),
wrap(0, 1);
return ret;
}
}
VVP reduceCH(VVP A) {
VVP ret; map<P3, VP> M;
for (VP nowF : A) {
P3 dir = cross(nowF[1] - nowF[0], nowF[2] - nowF[0]).unit();
for (P3 k1 : nowF) M[dir].pb(k1);
}
for (pair<P3, VP> nowF : M) ret.pb(convexHull2D(nowF.se, nowF.fi));
return ret;
}
// Biến một mặt thành dạng (điểm, vector pháp tuyến)
pair<P3, P3> getF(VP F) {
return mp(F[0], cross(F[1] - F[0], F[2] - F[0]).unit());
}
// Cắt 3D: giữ lại phần dot(dir,x-p)>=0
VVP ConvexCut3D(VVP A, P3 p, P3 dir) {
VVP ret; VP sec;
for (VP nowF : A) {
int n = nowF.size(); VP ans; int dif = 0;
for (int i = 0; i < n; i++) {
int d1 = sign(dot(dir, nowF[i] - p));
int d2 = sign(dot(dir, nowF[(i + 1) % n] - p));
if (d1 >= 0) ans.pb(nowF[i]);
if (d1 * d2 < 0) {
P3 q = getFL(p, dir, nowF[i], nowF[(i + 1) % n])[0];
ans.push_back(q); sec.push_back(q);
}
if (d1 == 0) sec.push_back(nowF[i]); else dif = 1;
dif |= (sign(dot(dir, cross(nowF[(i + 1) % n] - nowF[i], nowF[(i + 1) % n] - nowF[i]))) == -1);
}
if (ans.size() > 0 && dif) ret.push_back(ans);
}
if (sec.size() > 0) ret.push_back(convexHull2D(sec, dir));
return ret;
}
}
db vol(VVP A) {
if (A.size() == 0) return 0; P3 p = A[0][0]; db ans = 0;
for (VP nowF : A)
for (int i = 2; i < nowF.size(); i++)

```



```

    ans += abs(getV(p, nowF[0], nowF[i - 1], nowF[i]));
    return ans / 6;
}
VVP init(db INF) {
    VVP pss(6, VP(4));
    pss[0][0] = pss[1][0] = pss[2][0] = { -INF, -INF, -INF};
    pss[0][3] = pss[1][1] = pss[5][2] = { -INF, -INF, INF};
    pss[0][1] = pss[2][3] = pss[4][2] = { -INF, INF, -INF};
    pss[0][2] = pss[5][3] = pss[4][1] = { -INF, INF, INF};
    pss[1][3] = pss[2][1] = pss[3][2] = {INF, -INF, -INF};
    pss[1][2] = pss[5][1] = pss[3][3] = {INF, -INF, INF};
    pss[2][2] = pss[4][3] = pss[3][1] = {INF, INF, -INF};
    pss[5][0] = pss[4][0] = pss[3][0] = {INF, INF, INF};
    return pss;
}

```